the Red Hat Software Collections. Red Hat and Microsoft collaborate to ensure that .NET Core works well on RHEL.

Comparisons with Mono

Mono is the original cross-platform and open source .NET implementation, and it was first shipped in 2004. It can be thought of as a community clone of the .NET Framework. The Mono project team relied on the open .NET standards (notably ECMA 335) published by Microsoft to provide a compatible implementation.

The major differences between .NET Core and Mono include the following.

- *App-models:* Mono supports a subset of the .NET Framework app-models (for example, Windows Forms) and some additional ones (like Xamarin.iOS) through the Xamarin product. .NET Core doesn't support these.
- *APIs:* Mono supports a large subset of the .NET Framework APIs, using the same assembly names and factoring.
- *Platforms:* Mono supports many platforms and CPUs.
- *Open source:* Mono and .NET Core both use the MIT licence and are .NET Foundation projects.
- *Focus:* The primary focus of Mono in recent years has been mobile platforms, while .NET Core is focused on the cloud and desktop workloads.
- *Mobile:* Xamarin/Mono is a .NET implementation for running apps on all the major mobile operating systems.

Languages

C#, Visual Basic and F# languages can be used to write applications and libraries for .NET Core. These languages are or can be integrated into your favourite text editors and IDEs, including Visual Studio, Visual Studio Code, Sublime Text and Vim. This integration is provided, in part, by the good folks from the OmniSharp and Ionide projects.

Comparing .NET Core with .NET Framework

There are fundamental differences between the two and your choice depends on what you want to accomplish. Table 2 guides you on when to use each.

Creating a 'Hello World' application using .NET Core

Let us look at how to create our first *Hello World* application using .NET Core in Ubuntu. You can either use the Visual Studio code editor or the command line to write this code. For the sake of simplicity, I am using the command line here.

The prerequisites are:

- .NET Core SDK 2.1
- A text editor or code editor of your choice

I am assuming here that you have Ubuntu 18 for this hands-on tutorial. If you have something else, then please go to *https://www.microsoft.com/net/download* to get OS-specific SDK installation instructions.

Step 1: Register Microsoft key and feed: Before installing .NET, we'll need to register the Microsoft key and the product repository, and install the required dependencies. This only needs to be done once, per machine. Open a command prompt and run the following commands:

```
$ wget -q https://packages.microsoft.com/config/ubuntu/18.04/
packages-microsoft-prod.deb
$ sudo dpkg -i packages-microsoft-prod.deb
```

***Step 2: Install .NET SDK*:** Update the products available for installation, then install the .NET SDK. In your command prompt, run the following commands:

```
$ sudo apt-get install apt-transport-https
$ sudo apt-get update
$ sudo apt-get install dotnet-sdk-2.1
```

Step 3: Hello, console app! Open a terminal window and create a folder named *Hello*. Navigate to the folder we've created and type the following command:

```
$ dotnet new console
$ dotnet run
```

Let's do a quick walkthrough of these commands. The first command is:

```
$ dotnet new console
```

dotnet new creates an up-to-date *Hello.csproj* project file with the dependencies necessary to build a console app. It also creates *Program.cs*, a basic file containing the entry point for the application.

Table 2: A comparison of .NET Core and .NET Framework

.NET Core for server apps	.NET Framework for server apps
• When you have cross-platform needs • If you are targeting microservices • If you are using Docker containers • If you need high-performance and scalable systems • If you need side-by-side .NET versions, per application	• If your app currently uses .NET Framework (it is recommended that you extend instead of migrating) • If your app uses third-party .NET libraries or NuGet packages are not available for .NET Core • If your app uses .NET technologies that aren't available for .NET Core • If your app uses a platform that doesn't support .NET Core